1. Feladat – Környezet előkészítése és adatimport

a) Csatlakozás

```
mongosh
use vendeglatas
```

b) JSON import

```
mongoimport --db vendeglatas --collection ettermek --file ettermek.json --jsonArray
// Vagy a mongotoolkit helyett
const data = JSON.parse(fs.readFileSync("ettermek.json", "utf8"))
db.ettermek.insertMany(data)
```

c) Ellenőrzés és kézi beszúrás

```
// Ellenorzes
db.ettermek.find()
db.szakacsok.find()
show collections

// 1-1 uj dokumentum beszurasa
db.ettermek.insertOne({
  nev: "Borkonyha",
  varos: "Budapest",
  cim: { utca: "Sas", hazszam: "3" },
  csillag: 5,
  specialitasok: ["modern magyar", "fine dining"]
})

db.szakacsok.insertOne({
  nev: "Szabo Anna",
  reszleg: "Garde Manger",
  eletkor: 32,
  fizetes: 400000,
  etterem_nev: "Borkonyha",
  vegzettsegek: ["Egyetem", "Le Cordon Bleu"]
})
```

2. Feladat – Lekérdezések és DML

a) Projekció

```
db.ettermek.find({}, { nev: 1, csillag: 1, _id: 0 })
```

b) Legalább 4 csillagos

```
db.ettermek.find({ csillag: { $gte: 4 } })
```

c) Budapest VAGY 5 csillagos

```
db.ettermek.find({
  $or: [{ varos: "Budapest" }, { csillag: 5 }]
})
```

d) Top 3 csillagszám szerint

```
db.ettermek.find().sort({ csillag: -1 }).limit(3)
```

e) 25–40 éves szakácsok

```
db.szakacsok.find({ eletkor: { $gte: 25, $lte: 40 } })
```

f) Csillagszám módosítása

```
db.ettermek.updateOne(
  { nev: "Anyukam Mondta" },
  { $set: { csillag: 5 } }
)
```

g) Új specialitás hozzáadása

```
db.ettermek.updateOne(
  { nev: "Trofea" },
  { $push: { specialitasok: "steak" } }
)
```

h) 3 csillag alatti éttermek törlése

```
db.ettermek.deleteMany({ csillag: { $lt: 3 } })
db.ettermek.countDocuments()
```

3. Feladat – Indexek

a) Explain index nélkül

```
db.ettermek.find({ nev: "Gundel" }).explain("executionStats")
```

b) Index létrehozása

```
db.ettermek.createIndex({ nev: 1 })
```

c) Explain index után

```
db.ettermek.find({ nev: "Gundel" }).explain("executionStats")
```

d) Összetett index

```
db.ettermek.createIndex({ varos: 1, csillag: -1 })
```

e) Listázás és törlés

```
db.ettermek.getIndexes()

db.ettermek.dropIndex("nev_1")
```

4. Feladat – Aggregációs pipeline

a) Éttermek városonként + átlagos csillagszám

```
db.ettermek.aggregate([
  {
    $group: {
      _id: "$varos",
      db_etterem: { $sum: 1 },
      atlag_csillag: { $avg: "$csillag" }
    }
  },
  { $sort: { atlag_csillag: -1 } }
])
```

b) $unwind – végzettségek kibontása

```
db.szakacsok.aggregate([
  { $unwind: "$vegzettsegek" },
  {
    $group: {
      _id: "$vegzettsegek",
      szakacsok_szama: { $sum: 1 }
    }
  },
  { $sort: { szakacsok_szama: -1 } }
])
```

c) $lookup + szűrés 4+ csillagos éttermekre

```
db.szakacsok.aggregate([
  {
    $lookup: {
      from: "ettermek",
      localField: "etterem_nev",
      foreignField: "nev",
      as: "etterem_adatok"
    }
  },
  { $unwind: "$etterem_adatok" },
  { $match: { "etterem_adatok.csillag": { $gte: 4 } } },
  {
    $project: {
```

```
      _id: 0,
      nev: 1,
      reszleg: 1,
      etterem: "$etterem_adatok.nev",
      csillag: "$etterem_adatok.csillag"
    }
  }
])
```

Megjegyzés: A `$lookup` eredménye tömb – az `$unwind` kibontja, utána a beágyazott mezőkre közvetlenül szűrhetünk.

5. Feladat – Legidősebb szakács éttermenként

```
db.szakacsok.aggregate([
  { $sort: { eletkor: -1 } },
  {
    $group: {
      _id: "$etterem_nev",
      legidosebb_nev: { $first: "$nev" },
      legidosebb_kor: { $first: "$eletkor" }
    }
  }
])
```

A `$sort` + `$group` + `$first` minta: először rendezünk, a `$first` az első (legidősebb) értéket veszi csoportonként. SQL analógia: `WHERE eletkor = (SELECT MAX(...))`.